

Here is the challenge again:

Write a generator `flatten(nested_list)` that flattens `[1, [2, [3, 4]], 5, [6]]` into `1, 2, 3, 4, 5, 6` **without recursion**, using a **stack**.

Part 1: How the Stack Code Actually Works (Step-by-Step)

Let's look at the code again:

```
python
def flatten(nested_list):
    stack = [nested_list] # We put the whole list onto our "to-do" pile

    while stack:          # While the pile is not empty
        item = stack.pop() # Take the top piece off the pile

        if isinstance(item, list): # If it's a list, we have to open it
            # Push the contents backwards so the leftmost comes out first
            stack.extend(reversed(item))
        else:
            yield item # If it's a number, give it back
```

Let's trace it with `[1, [2, 3]]`:

Step	What stack looks like	We pop() this	Action	What we yield
Start	<code>[[1, [2, 3]]]</code>	(nothing yet)	(start)	-
1	<code>[[1, [2, 3]]]</code>	<code>[1, [2, 3]]</code>	It's a list! We reverse it -> <code>[[2, 3], 1]</code> and push both. Now stack = <code>[[2, 3], 1]</code>	(nothing)
2	<code>[[2, 3], 1]</code>	<code>1</code>	It's NOT a list!	Yield 1
3	<code>[[2, 3]]</code>	<code>[2, 3]</code>	It's a list! Reverse -> <code>[3, 2]</code> and push. Stack = <code>[3, 2]</code>	(nothing)
4	<code>[3, 2]</code>	<code>2</code>	Not a list!	Yield 2

Step	What <code>stack</code> looks like	We <code>pop()</code> this	Action	What we <code>yield</code>
5	[3]	3	Not a list!	Yield 3

Final output: 1, 2, 3. Perfect order!

Part 2: Why do we use `reversed(item)`?

Because a **stack** works like a **spring-loaded pile of plates** (Last In, First Out). If you push 1 then 2 onto a stack, you pop 2 first.

- We want to process left-to-right (index 0, then index 1).
 - So, we push the items **backwards** (right-to-left).
 - That way, the leftmost item is sitting on the top of the pile, waiting to be popped first.
-

Part 3: The Memory Question (Why does recursion fail at 10,000 levels?)

The question asks: "What happens to memory usage if you used recursion for a list nested 10,000 levels deep? Why does the iterative version avoid this?"

Let me explain this like a story:

The Recursive Way (Bad for deep stuff):

Imagine you are looking for a toy inside a box. But inside that box is another box. Inside that one is another box. And so on, **10,000 times**.

With recursion, you open the first box, see a smaller box, and **you hold the first box open in your hands** while you open the second. Then you hold the second box open while you open the third.

By the time you reach box #10,000, you are physically holding **10,000 open boxes** in your arms!

Python has a strict limit (usually 1,000 boxes) because holding all those open boxes crashes your computer's memory (stack overflow).

The Iterative Stack Way (Good for deep stuff):

With our code, we don't hold boxes open. We just write down a **"To-Do List"** on a piece of paper.

If we find a box, we don't open it immediately and wait. Instead, we write on our paper: "Open box #2 later", and we put that paper in our backpack.

If we go 10,000 levels deep, our backpack just has 10,000 sticky notes in it. That takes up barely any space, and Python doesn't care how many sticky notes you have, as long as your computer has RAM.

To summarize the answer:

1. **Recursion** uses the **Call Stack** (a tiny, limited memory space). At 10,000 levels, it crashes with a `RecursionError` because it runs out of space holding all the unfinished function calls.

2. **Iterative (our code)** uses the **Heap** (a huge, flexible memory space) to store our own `stack` list. It never holds function calls waiting for an answer, so it safely handles 10,000 levels without crashing.